

MySQL & Redis 面试睡前播客复习（28问）

覆盖高频基础 + 原理 + 追问方向（适合朗读）

生成日期：2026-02-07 | 使用方式：建议按“问题 -> 回答 -> 追问”朗读，每晚复习5-8题。

今晚复习路线（建议）	时长
先MySQL：索引/事务/锁/日志（Q1-Q12）	约15-20分钟
再Redis：数据结构/持久化/复制/集群/锁（Q13-Q25）	约20-25分钟
最后综合：缓存问题/一致性/秒杀（Q26-Q28）	约8-12分钟

MySQL 高频 12 问

Q1. SQL vs NoSQL：什么时候选MySQL，什么时候选Redis/Mongo？

- MySQL是关系型数据库，核心优势是强事务能力（ACID）和复杂查询；Redis属于NoSQL里的内存KV，强调性能和高并发，通常用于缓存、计数、队列、分布式锁等。

- 选型可用一句话：需要强一致 + 复杂关系/查询 ->

MySQL；读多写少、热点明显、可接受最终一致 -> Redis做缓存。

面试官追问：

面试追问：什么是ACID？什么是BASE？哪些业务必须ACID（如资金、库存扣减）？

Q2. InnoDB为什么是默认引擎？和MyISAM的差异抓哪几点？

- InnoDB支持事务、行级锁、崩溃恢复（redo log）、外键；MyISAM不支持事务、以表锁为主、崩溃恢复能力弱（但某些只读场景可能更快）。

- 面试最常考：事务与锁粒度（行锁 vs 表锁）、崩溃恢复（redo/undo）、并发性能。

面试官追问：面试追问：InnoDB行锁是怎么实现的（索引上加锁）？为什么没有索引会锁全表？

Q3. 一条SELECT语句的执行顺序是什么？（写出来就加分）

- 常见逻辑顺序：FROM -> JOIN/ON -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT。

- MySQL优化器会改写/重排执行计划，但你回答“逻辑顺序”即可。

面试官追问：面试追问：WHERE和HAVING区别？ORDER BY能否用索引？

Q4. 索引是什么？为什么MySQL常用B+Tree？

- 索引是为了减少磁盘IO和比较次数的“有序数据结构”。InnoDB二级索引叶子节点存主键，回表拿整行；覆盖索引可避免回表。

- B+Tree适合磁盘：分支因子大、高度低、范围查询友好。

面试官追问：面试追问：什么是回表？什么是覆盖索引？为什么联合索引更好？

Q5. 联合索引最左前缀怎么用？怎么避免失效？

- 联合索引 (a,b,c) 能支持按 a、(a,b)、(a,b,c) 的过滤/排序；若跳过a直接用b通常无法利用索引（最左前缀）。

- 失效高频点：对索引列做函数/隐式类型转换、前导模糊 like '%x'、范围条件后面的列难以继续利用。

面试官追问：面试追问：where a=1 and b>2 and c=3 怎么走？范围列之后还能用索引吗？

Q6. Explain最该看哪几个字段？看到type=ALL怎么办？

- 优先看：type（访问类型）、key（用到的索引）、rows（扫描行数）、Extra（Using index/Using filesort/Using temporary）。
- type=ALL通常是全表扫描：常见优化是补索引、改写条件、避免在索引列上做函数、让过滤条件更可用。

面试官追问：面试官追问：Using filesort一定慢吗？什么时候可接受？

Q7. 事务四大特性ACID分别怎么在InnoDB里落地？

- 原子性靠undo log（回滚）；持久性靠redo log（崩溃恢复）；隔离性靠MVCC或锁；一致性由原子性+隔离性+持久性共同保证。
- 记忆法：undo负责“撤销”，redo负责“重做”。

面试官追问：面试官追问：一致性为什么不是靠某个单点机制？

Q8. 隔离级别有哪些？各自解决了哪些并发问题？

- 四种：读未提交、读已提交、可重复读、串行化。
- 并发异常三件套：脏读、不可重复读、幻读。隔离越强并发越差。InnoDB在可重复读下配合MVCC和间隙锁可减少幻读风险。

面试官追问：面试官追问：MySQL默认隔离级别是什么？为什么？

Q9. MVCC是什么？ReadView/undo链在解决什么问题？

- MVCC用“多版本”让读不加锁：事务读到的是符合其可见性的版本。undo log里保留旧版本形成版本链；ReadView决定当前事务能看到哪些版本。
- 效果：提升并发读性能，减少读写互斥。

面试官追问：面试官追问：RC与RR下ReadView生成时机有什么差异？

Q10. InnoDB有哪些锁？间隙锁/next-key lock到底防什么？

- 行锁一般加在索引记录上；间隙锁锁住“范围空隙”；next-key lock=记录锁+间隙锁，用来防止范围条件下的幻读（插入新记录）。
- 想锁住读到的行并阻塞并发写：SELECT ... FOR UPDATE / LOCK IN SHARE MODE。

面试官追问：面试官追问：为什么“没走索引”会导致锁范围扩大甚至锁表？

Q11. redo log / undo log / binlog分别干嘛？为什么需要两阶段提交？

- redo log：InnoDB物理日志，WAL保证崩溃恢复；undo log：逻辑回滚/版本；binlog：Server层逻辑日志，用于主从复制/备份恢复。

•

两阶段提交把redo与binlog的落盘协调起来，避免“数据写进引擎但binlog没记”或反过来的不一致。

面试官追问：面试官追问：主从复制靠什么日志？为什么binlog是逻辑日志？

Q12. double write buffer是解决什么问题？为什么redo log不够？

- MySQL页默认16KB，写盘会拆成多个OS页（常见4KB），中途断电会造成“页写一半”导致数据页损坏。
- double write先把页写到双写区，确认完整后再写真实数据文件；页已损坏时redo无法修复“坏页本身”，需要副本来恢复。

面试官追问：面试官追问：double write的性能影响？为什么还能接受？

Redis 高频 13 问

Q13. Redis为什么快？“单线程”到底单在哪里？

- Redis主要操作在内存完成，瓶颈常在内存/网络而非CPU；单线程避免锁竞争和线程切换；配合IO多路复用（select/epoll）一线程处理大量连接。
- Redis并非完全单线程：后台BIO线程负责AOF刷盘、关闭文件、lazyfree等；Redis 6.0以后引入IO线程并行处理网络读写，但命令执行仍以单线程为主。

面试官追问：面试追问：什么情况下Redis会被慢命令拖垮？如何避免大key/慢查询？

Q14. Redis有哪些数据类型？各自常见业务场景是什么？

- 基础5种：String、Hash、List、Set、Zset；扩展如BitMap/HyperLogLog/GEO/Stream等。
- 面试说法：String做计数/分布式锁；Hash做对象/购物车；List做队列；Set做去重/共同关注；Zset做排行榜/范围查询。

面试官追问：面试追问：为什么Zset适合排行榜？有哪些命令？

Q15. SDS为什么比C字符串好？

- SDS通过len字段让长度获取O(1)，支持二进制安全（不依赖\0结束），并通过alloc/len避免缓冲区溢出，空间不足可自动扩容。

面试官追问：面试追问：SDS的扩容策略？会不会频繁拷贝？

Q16. 为什么Zset用跳表而不是B+Tree？

- Redis是内存场景，多几次指针跳转开销很小；跳表实现简单、写操作局部修改不需要复杂分裂/合并，性能达到O(logN)且代码可维护性强。
- B+Tree更适合磁盘场景（减少IO），在内存里复杂度收益不大。

面试官追问：面试追问：跳表层高怎么随机生成？复杂度为什么是logN？

Q17. Redis哈希表怎么扩容？“渐进式rehash”在干嘛？

- rehash迁移会很耗时，Redis采用渐进式：每次对哈希表进行增删改查时，顺带搬迁一小部分桶，从而把一次性大停顿分摊到多次请求中。
- rehash期间查找会先查旧表再查新表；新增写入新表。

面试官追问：面试追问：渐进式rehash期间会不会数据不一致？为什么不会？

Q18. 持久化：RDB vs AOF各自优缺点？AOF三种刷盘策略怎么选？

- RDB是某一时刻的快照，文件小、恢复快，但可能丢快照间数据；AOF记录写命令，数据更安全但文件更大，需要重写。
- AOF刷盘：Always最安全但慢；Everysec性能/安全折中（最多丢1秒）；No交给OS风险更大。

面试官追问：面试追问：AOF重写为什么不会阻塞主线程？会用到子进程吗？

Q19. 过期删除 vs 内存淘汰：区别一句话说明白

- 过期删除处理“已经过期的key”——Redis采用惰性删除+定期删除；内存淘汰是在达到maxmemory时决定“删谁”以释放空间。

面试官追问：面试追问：定期删除为什么是随机抽样而不是全量扫描？

Q20. 常见内存淘汰策略怎么背？（volatile vs allkeys）

- noeviction是不淘汰直接报错；其余大体分两类：只在设置了TTL的key里淘汰（volatile-*）和在所有key里淘汰（allkeys-*），再按LRU/LFU/Random/TTL等规则选择。

面试官追问：面试追问：热点缓存更适合allkeys-lru还是volatile-lru？为什么？

Q21. 主从复制：全量同步怎么走？增量同步靠什么续传？

- 全量同步：主生成RDB->传给从->从清空加载；期间主把新写命令写进replication backlog buffer，RDB传完后再补发。
- 增量同步：从用PSYNC带runID+offset续传，主用repl_backlog_buffer和复制偏移量决定能否增量，否则回退到全量。

面试官追问：面试追问：repl_backlog_buffer太小会怎样？怎么调？

Q22. 哨兵解决了什么问题？选主的核心依据是什么？

- 哨兵负责监控、故障转移、通知：主挂了会选一个从升级为主，并通知其它从与客户端更新地址。

- 选主常见优先级：过滤故障节点->优先级(slave-priority)->复制偏移量最大->runid最小。

面试官追问：面试追问：主观下线 vs 客观下线差异？quorum的作用？

Q23. Redis Cluster为什么是16384槽？客户端如何路由？

- 集群用哈希槽分片：key做CRC16后对16384取模定位槽位，槽位再映射到节点。
- 客户端启动会拉取并缓存槽位-节点映射（如CLUSTER SLOTS），按槽位直连目标节点；遇到迁移会收到MOVED/ASK重定向并更新缓存。

面试官追问：面试追问：为什么跨槽多键操作受限？如何用hash tag让key落同槽？

Q24. Redis事务能回滚吗？怎么实现“真正原子性”？

- MULTI/EXEC能把多条命令打包执行，但命令错误不会回滚；要保证多步操作原子性，优先用Lua脚本把逻辑写成一个脚本整体执行。

面试官追问：面试追问：为什么Lua脚本天然原子？会不会阻塞？

Q25. 分布式锁怎么用Redis实现？为什么要用Lua解锁？

- 加锁：SET lock_key unique_value NX PX 10000（不存在才设置+过期时间防死锁）。
- 解锁要先比对unique_value再DEL，这两步必须原子，否则会误删别人的锁，所以用Lua脚本。

面试官追问：面试官追问：锁续期怎么做？RedLock争议点你怎么看？

综合与系统设计 3 问

Q26. 缓存雪崩/击穿/穿透：各自是什么？怎么解决？

- 雪崩：大量key同一时间过期或Redis宕机 -> DB被打爆；解法：过期时间加随机、热点永不过期+后台刷新、限流降级等。
 - 击穿：热点key过期 -> 高并发打到DB；解法：互斥锁/单飞、热点永不过期。
 - 穿透：查不存在数据 -> 缓存/DB都没有；解法：参数校验、缓存空值、布隆过滤器。
- 面试官追问：面试官追问：布隆过滤器的误判是什么？能删除元素吗？

Q27. 如何保证MySQL和Redis缓存一致性？（旁路缓存）

- 读：缓存未命中再读DB并回填缓存。写：先更新DB，再删除缓存（而不是更新缓存），降低并发写下的复杂性。
 - 删除缓存失败可用重试（消息队列）或订阅binlog变更再删缓存等方案提升最终一致性。
- 面试官追问：面试官追问：为什么是“更新DB后删缓存”，不是先删后更？并发下会发生什么？

Q28. 高并发秒杀怎么设计避免超卖？MySQL锁 vs Redis预扣减怎么取舍？

- 数据库方案：事务里SELECT ... FOR UPDATE锁行，或UPDATE ... WHERE stock>0 以条件约束库存。
 - 缓存方案：Redis用DECR原子预扣减，库存不足直接失败；把下单写库放到异步队列里，削峰填谷。
- 面试官追问：面试官追问：Redis预扣减成功但写库失败如何补偿？如何保证最终一致？

资料来源（融合整理）

- 1) 《MySQL面试题_QA整理》（2026年1月版，来源：小林coding）
- 2) 《Redis面试题_QA整理》（2026年1月版，来源：小林coding）

说明：本文为学习与面试复习用途的二次整理，建议结合实际项目场景举例。